

Chapter 5: JavaScript and HTML Documents

Comprehensive Exam Preparation Notes

Based on: Programming the World Wide Web – Robert W. Sebesta

 **NOTE:** These notes cover ALL topics from Chapter 5: JavaScript Execution Environment, DOM, Object Hierarchy, Element Access, Events & Event Handling, DOM 2 Event Model, Navigator Object, and DOM Tree Traversal/Modification.

5.1 JavaScript Execution Environment

What is the Execution Environment? JavaScript runs inside a web browser on the client side. The browser provides a structured environment where scripts interact with the displayed document through a set of built-in objects.

The Window Object

- Represents the browser window or tab displaying a document
- All properties of the Window object are globally visible to all JavaScripts running in that window
- Global variables declared in JavaScript automatically become properties of the Window object
- There can be more than one Window object (e.g., when popups or frames are used)
- Acts as the top-level container for all other objects in the JavaScript environment

The Document Object

- Represents the HTML/XHTML document currently displayed in the browser window
- Accessed via the document property of the Window object: window.document or simply document

- Contains property arrays for accessing page elements: forms, links, anchors, images, etc.

 **NOTE:** The *Window* and *Document* objects form the foundation of client-side JavaScript. Every script runs within a window, and every page element is accessible through the document.

5.2 Object Hierarchy and the Document Object Model (DOM)

Object Hierarchy Overview

The browser's objects are organized in a strict hierarchy. At the top is the **Window** object, which contains **History**, **Location**, and **Document**. The Document contains Links, Anchors, and Forms. Each Form contains form elements like Text, Radio, Checkbox, Textarea, Password, Select (with Options), Button, Reset, and Submit.

DOM Levels

The DOM evolved through several official levels:

1. DOM Level 0 — Informal, early browser-specific implementation. Not a W3C standard but widely implemented.
2. DOM Level 1 — Standardized XHTML/XML document structure. Defined basic tree traversal and manipulation.
3. DOM Level 2 — Added event model, support for XML namespaces and CSS. More powerful and modular.
4. DOM Level 3 — Support for XPath and keyboard event handling. Interface for serializing documents as XML.

What is the DOM?

- DOM stands for Document Object Model
- It is an API (Application Programming Interface) that defines how application programs interact with XHTML/HTML documents
- Each language (JavaScript, Java, etc.) must define its own binding to the DOM interfaces
- DOM specification consists of a collection of interfaces, with one for each type of document tree node
- The DOM describes an abstract model of a document as a tree of nodes

DOM as an Abstract Model

- Interfaces describe methods and properties in a language-neutral way
- The interfaces describe a tree structure representing the document
- In JavaScript, data are called properties and operations are called methods

Example: The HTML tag `<input type="text" name="address">` is represented as an object with two properties: `type = "text"` and `name = "address"`

DOM Tree Structure Example

Consider a simple HTML document with a table. The DOM represents it as a tree:

- Document is the root node
- head and body are children of Document
- title is a child of head; the title text is a child of title
- table is a child of body
- Each tr row is a child of table
- Each th and td is a child of a tr
- Text content is a child (text node) of th/td

 **NOTE:** Key concept: Attributes of HTML elements become named properties of element node objects in JavaScript.

5.3 Element Access in JavaScript

To manipulate or respond to HTML elements, JavaScript must first obtain a reference (DOM address) to the corresponding object. There are several methods.

Method 1: Using the `forms[]` and `elements[]` Array (DOM Level 0)

- The Window object provides `document.forms[]` — an array of all form elements
- Each form has an `elements[]` array for its child inputs

Example:

```
<form action=""><input type="button" name="turnItOn"></form>
var dom = document.forms[0].elements[0];
```

Disadvantage: The index is position-dependent. If a new element is added before the target element, the index changes and the reference breaks.

Method 2: Using name Attributes

- Give the form a name attribute, and also name the input
- The XHTML elements are then accessible by their names

Example:

```
<form name="myForm" action=""><input type="button"
name="rdType"></form>
var dom = document.myForm.rdType;
```

Disadvantage: The XHTML 1.1 standard does not allow the name attribute on the form element itself, causing validation problems.

Method 3: Using the id Attribute with `getElementById` (Recommended)

- Assign a unique id to the element
- Use document.getElementById() to retrieve a reference
- id values must be unique across the entire document

Example:

```
<form action=""><input type="button" id="turnItOn"></form>
var dom = document.getElementById("turnItOn");
```

This is the preferred and most reliable method.

Method 4: Implicit Arrays for Checkboxes and Radio Buttons

When multiple checkboxes or radio buttons share the same name, they form an implicit array that is a property of their containing form.

Example HTML:

```
<form id="vehicleGroup">
  <input type="checkbox" name="vehicle" value="car"/> Car
  <input type="checkbox" name="vehicle" value="truck"/> Truck
  <input type="checkbox" name="vehicle" value="bike"/> Bike
</form>
```

Counting checked items:

```
var numChecked = 0;
var dom = document.getElementById("vehicleGroup");
for(var i = 0; i < dom.vehicle.length; i++)
  if(dom.vehicle[i].checked) numChecked++;
```

 **NOTE:** The implicit array approach is the standard way to iterate through a group of related radio buttons or checkboxes. First get the form using getElementById, then access the named group as a property.

5.4 Events and Event Handling

What is Event-Driven Programming?

- A programming style where code segments called event handlers are written to be activated when certain events occur
- Events represent activity in the environment, especially user actions like mouse movements or keyboard input
- An event handler is a script designed to execute when a specific event occurs
- Most common use: validating form data on the client side to save round-trips to the server

Important Event Properties

- Events are JavaScript objects with case-sensitive names

- Example: click is a valid event, but Click is NOT a valid event
- Registration is the process of connecting an event handler to an event

Warning: Do NOT Use document.write() in Event Handlers

- Events occur after the entire document has already loaded and been displayed
- If document.write() is called in an event handler, the new content may overwrite the entire existing document
- Use DOM manipulation methods instead (see Section 5.10)

How a Text Element Gains Focus

A text element gains focus (becomes active) in three ways:

5. The user moves the mouse cursor over it and presses the left mouse button
6. The user presses the Tab key to navigate to the element
7. By calling the element's focus() method programmatically from JavaScript

Losing focus is called blurring (the blur event fires).

Events, Their Attributes, and Associated Tags

Event	HTML Attribute
blur	onblur
change	onchange
click	onclick
focus	onfocus
load	onload
mousedown	onmousedown
mousemove	onmousemove
mouseout	onmouseout
mouseover	onmouseover
mouseup	onmouseup
select	onselect
submit	onsubmit
unload	onunload
dblclick	ondblclick
keydown	onkeydown
keypress	onkeypress
keyup	onkeyup

Event Attribute Details (Table 5.2 Summary)

Attribute	Tag & Description
onblur	<a>, <button>, <input>, <textarea>, <select> — Element loses input focus
onchange	<input>, <textarea>, <select> — Element is changed AND loses focus
onclick	<a>, <input> — User clicks the element or link
ondblclick	Most elements — User double-clicks the left mouse button
onfocus	<a>, <input>, <textarea>, <select> — Element receives input focus
onkeydown	<body>, form elements — A key is pressed down
onkeypress	<body>, form elements — A key is pressed down and released
onkeyup	<body>, form elements — A key is released
onload	<body> — The document has finished loading
onsubmit	<form> — The form is submitted
onunload	<body> — The document is being unloaded (navigating away)

Event Handler Registration — DOM Level 0

Method 1: Assign handler directly as a tag attribute (inline)

```
<input type="button" name="myButton" onclick="alert('You clicked!')"/>
```

Method 2: Use a function call in the attribute (for longer handlers)

```
<input type="button" name="myButton" onclick="myHandler();"/>
```

Method 3: Assign to the node property using JavaScript

```
document.getElementById("myButton").onclick = myHandler;
```

 **NOTE:** Important: When assigning to a node property, write `myHandler` (no parentheses). Writing `myHandler()` calls the function immediately instead of registering it as a handler.

Comparing the Registration Methods

Method	Characteristics
Attribute assignment (inline)	More flexible — allows passing parameters to the function without wrapping in anonymous function
Node property assignment	Cleaner — separates HTML and JavaScript code. Allows changing the handler dynamically later.

5.5 Handling Events from Body Elements

The **<body>** element most commonly creates two events:

- load — fired when the page has fully loaded
- unload — fired when the user navigates away or closes the page

onload Event

- Used via the onload attribute of <body>
- Allows a script to execute initialization code as soon as the page loads
- Common use: set up dynamic content, start timers, pre-populate fields

Example: `<body onload="initialize();">`

onunload Event

- Used via the onunload attribute of <body>
- Used for cleanup tasks before the document is unloaded
- Classic example: if the document opened a popup window, the onunload handler can close it

5.6 Handling Events from Button Elements

Registering Button Events

Given the button: `<input type="button" name="freeOffer" id="freeButton"/>`

Option 1 — via event attribute:

```
<input type="button" id="freeButton" onclick="freeButtonHandler();" />
```

Option 2 — via node property:

```
document.getElementById("freeButton").onclick = freeButtonHandler;
```

 **NOTE:** The property assignment statement should come *AFTER* both: (1) the handler function definition, and (2) the HTML form has been parsed. This ensures JavaScript has seen both before the assignment.

Checkboxes and Radio Buttons

- A parameter can be passed to the handler when registering via attribute (e.g., passing the value to identify which button was clicked)
- When registering via node property (assigning a reference), NO parameters are passed when JavaScript calls the function
- In this case, the handler code must identify which element triggered the event using the this keyword or event properties

5.7 Handling Events from Text Box and Password Elements

Events Generated by Text Box and Password Fields

Four events can be generated:

8. blur — fires when the element loses focus
9. focus — fires when the element gains focus
10. change — fires when content is changed and then focus moves away
11. select — fires when the user selects text inside the input

Preventing Edits (nochange technique)

- You can prevent a user from modifying a field by blurring it immediately whenever it gains focus
- The onfocus handler calls blur() on the same element
- This makes the field effectively read-only while keeping it visible

Validating Form Input

- Client-side validation using JavaScript provides faster feedback to the user
- Server-side validation requires a full round-trip and a new error page

Standard error-handling pattern:

12. Show an alert() message describing the validation error
 13. Call focus() on the problematic field to direct user attention
 14. Call select() to highlight the existing incorrect text for easy editing
- If an event handler returns false, the browser does NOT perform the default action
 - This is used on a Submit button's onclick handler — returning false prevents form submission if validation fails

Validating Input with Regular Expressions

Common validation patterns:

- Name format (First, Last, Middle-Initial each capitalized):

```
/^[A-Z][a-z]+, ?[A-Z][a-z]+, ?[A-Z]\.?$ /
```

- Phone format (ddd-ddd-dddd):

```
/^\d{3}-\d{3}-\d{4}$/
```

- The ^ and \$ anchors ensure no leading or trailing extra characters are accepted

 **NOTE:** Always use ^ and \$ anchors in validation regexes to ensure the ENTIRE input matches the pattern, not just a substring of it.

5.8 DOM 2 Event Model

Overview of DOM 2 Events

- DOM 2 is a modularized interface with an Events module that defines several submodules
- HTMLEvents and MouseEvent are the most commonly used submodules
- An event object is implicitly passed as a parameter to every event handler
- Properties of the event object provide details about what happened
- Mouse events extend the interface to include the mouse position at the time of the event

Drawback: Microsoft Internet Explorer historically did not support DOM 2 — this was a significant compatibility issue.

Event Propagation — The Three Phases

When an event occurs on an element (the **target node**), a three-phase process begins:

Phase 1: Capturing Phase

- Starts at the document root and travels DOWN to the target node
- Each node is examined in order from root to target
- If a node has a handler registered for capture (`capture=true`), it is executed
- Only handlers explicitly enabled for capture run in this phase

Phase 2: Target Phase

- All handlers registered on the target node itself are executed
- This happens regardless of capture setting

Phase 3: Bubbling Phase

- Starts from the target's parent and travels UP to the document root
- Any handler registered for the event on any ancestor node is executed
- Handlers run whether or not they were enabled for capture

Events that do NOT bubble: load, unload, blur, focus

Events that CAN bubble: All mouse events (click, mousedown, mouseup, mouseover, etc.)

Stopping propagation: A handler can call `event.stopPropagation()` to prevent further propagation at any phase.

Advantages of Event Propagation / Bubbling

- In large documents, you do NOT need to attach handlers to every individual element
- Event handling can be centralized at an ancestor node
- Example: A calculator has many digit buttons. Instead of adding a handler to each button, place ONE handler on their common parent container — all clicks will bubble up to it
- This significantly reduces code and improves maintainability

Preventing Default Browser Actions

- To prevent the browser's default action (e.g., submitting a form, following a link), call:

```
event.preventDefault();
```

Event Handler Registration in DOM 2

- Handlers are called listeners in DOM 2
- Use `addEventListener()` to register a handler — defined in the `EventTarget` interface

`addEventListener()` takes three parameters:

15. A string — the name of the event (e.g., "mouseup", "submit", "click")
16. The handler function — either the function code itself or the function name (reference)
17. A Boolean — true enables capture phase; false means bubbling phase only

Temporary Handlers

- Register a handler using `addEventListener()` only when needed
- Remove it using `removeEventListener()` when it is no longer required
- This enables dynamic, context-dependent event handling

Key Event Object Properties

Property	Description
<code>target</code>	References the original target node — the element where the event was created
<code>currentTarget</code>	References the node where the currently executing handler is registered
<code>clientX</code> / <code>clientY</code>	X and Y coordinates of the mouse cursor relative to the browser window (MouseEvent only)
<code>stopPropagation()</code>	Method to stop the event from propagating further through the tree
<code>preventDefault()</code>	Method to cancel the browser's default action for this event

 **NOTE:** *target vs currentTarget: target is always the element that was clicked/acted on. currentTarget is the element whose handler is currently running — these can differ when bubbling.*

5.9 The navigator Object

The **navigator** object is a property of the Window object. It provides information about the browser in which the script is executing.

Key Properties

Property	Description
----------	-------------

appName	Returns the name of the browser (e.g., 'Netscape' for Firefox, Chrome; 'Microsoft Internet Explorer' for IE)
appVersion	Returns the version string of the browser

 **NOTE:** Important quirk: Firefox and Chrome report their appName as 'Netscape' for historical compatibility reasons. Always test against actual version strings for reliable browser detection.

5.10 DOM Tree Traversal and Modification

Overview

- Every HTML element in a document has a corresponding Element object in the DOM
- These Element objects have properties and methods for navigating the tree (traversal) and changing the document (modification)

DOM Tree Traversal Properties

Each Element object exposes properties to navigate to related nodes:

Property	Description
parentNode	References the parent node of the current element
previousSibling	References the previous sibling node in the parent's list of children
nextSibling	References the next sibling node in the parent's list of children
firstChild	References the first child node of this element (could be a text node or element node)
lastChild	References the last child node of this element

Note: Children can be text nodes (containing text content) or element nodes (nested HTML elements).

DOM Tree Modification Methods

Method	Description
insertBefore(newChild, refChild)	Inserts newChild as a child of the target node, positioned before refChild
replaceChild(newChild, oldChild)	Replaces oldChild with newChild within the target node
removeChild(child)	Removes the specified child node from the target node
appendChild(newChild)	Adds newChild as the LAST child of the target node

 **NOTE:** These four methods (insertBefore, replaceChild, removeChild, appendChild) are the core tools for dynamically modifying page content. They enable rich interactive web applications without page reloads.

Quick Reference Summary for Exams

Key Concepts at a Glance

Topic	Key Point
DOM Origin	Originated to allow JavaScript/Java to be portable across browsers
Window Object	Top-level container; global variables are its properties
Document Object	Accessed as window.document; represents the displayed page
DOM 0	Informal, early browser model using forms[] and elements[] arrays
DOM 1	Standardized XHTML/XML document tree structure
DOM 2	Adds formal event model, CSS support, XML namespaces
DOM 3	Adds XPath support, keyboard events, XML serialization
getElementById	Preferred way to access elements — uses unique id attribute
Event registration DOM0	Via tag attribute OR via node.onclick = handler assignment
Event registration DOM2	Via addEventListener(event, handler, useCapture)
Capturing Phase	Root → target: only capture-enabled handlers fire
Bubbling Phase	Target parent → root: all registered handlers fire
Non-bubbling events	load, unload, blur, focus
stopPropagation()	Stops event from traveling further through the tree
preventDefault()	Stops browser's default action (e.g., form submit)
navigator.appName	Returns browser name; Firefox/Chrome report 'Netscape'
document.write() in handlers	NEVER use — overwrites existing document content
Validation return false	Returning false from submit handler prevents submission
appendChild	Adds node as LAST child
insertBefore	Adds node BEFORE a reference child
removeChild	Removes a specific child node
replaceChild	Swaps old child with new child